# Intro to DL4MSc - Input Normalization and Data Augmentation

Alexander Binder

November 17, 2025

know where to look for

- Chapter 14.1 in https://d2l.ai/d2l-en.pdf
- https://pytorch.org/

## Takeaway points

at the end of this lecture you should be able to:

- input normalization
- be able to give suggestions how to use images of varying sizes in a neural network with a fixed input size
- be able to use a neural network with a different input size than the one used at training time

## Takeaway points

at the end of this lecture you should be able to:

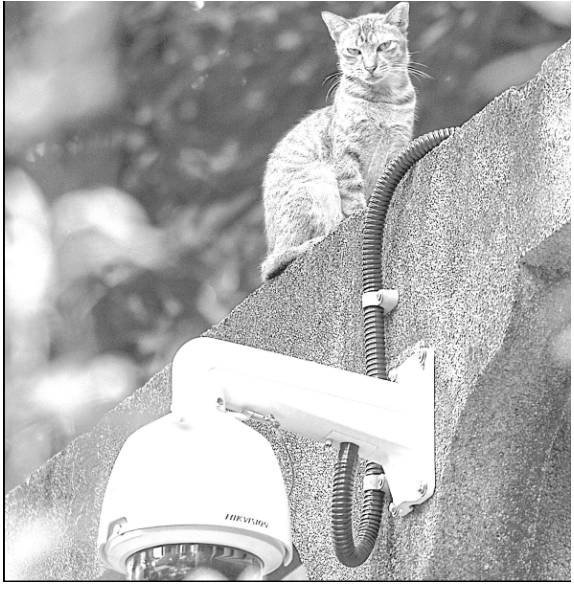
- explain some example data augmentations
- geometric versus photometric augmentation
- explain how to choose a data augmentation parameter.
- it is not a vision-only approach

# Motivating example

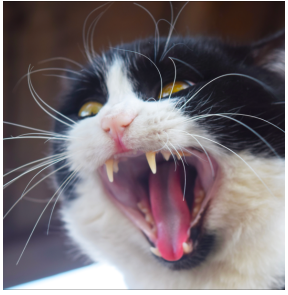
- A cat owner tasks you with training a network for classication, and provides you with a labeled dataset of images of (their) cats.
- They want to use the classifier to detect cats in their backyard with a low resolution infrared surveillance camera.

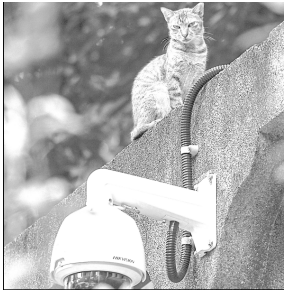
# Motivating example

- A cat owner tasks you with training a network for classication, and provides you with a labeled dataset of images of (their) cats.
- They want to use the classifier to detect cats in their backyard with a low resolution infrared surveillance camera.
- You notice that all the images are closeup indoor images of the (reluctant) faces of cats taken by the owner using an SLR.
- Do you expect any problems with this setup?



- Do you expect any problems with this setup?
- the probability distribution of train and test data is very different (cat sizes, color, resolution, lighting)
- we will talk about data normalization and data augmentation to reduce the impact of this





he goal of this lecture is to give an introduction to data augmentation.

- **Problem 1: Input normalization** to match the mean and standard deviation of values to how the network was trained
- **Problem 2: The input dimensionality** of a neural network may not fit the dimensionality of the input data. We discuss ways to deal with it.
- **Problem 3: use neural network with a different than the specified input size** – yes one can!!

- **Problem 4: Data augmentation at Testing time**
  allow the neural net to see *multiple views* of the same sample. One creates multiple views of one sample. The final prediction will be an average of the prediction scores, averaged over all views. "Look at all aspects to understand a thing."

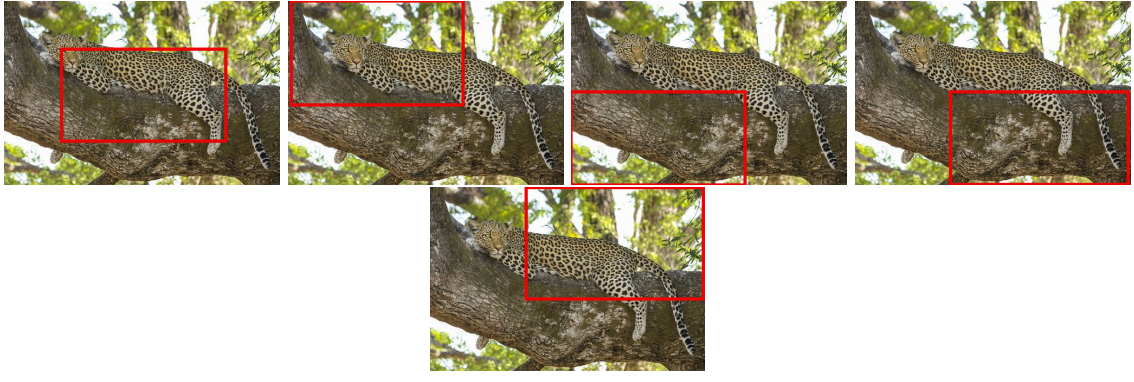
Example in this class: Do not input an image as a whole,

Original Image

- but input 5 crops: a cropped image from the center, and 4 corner crops.
- compute the prediction as an average over crops:

- example: use 5 crops: a cropped image from the center, and 4 corner crops.
- compute the prediction as an average of predictions $f(\cdot)$ over crops:

prediction $= \frac{1}{5}$( f(Center crop)+ f(upper left crop)+ f(lower left )+ f(lower right)+ f(lower right crop) )



- **Problem 5: Data augmentation at Training time:** allow the neural net to be trained with more data samples. Data augmentation increases the data size by creating many similar samples from one sample.
- Example: training with slight photometric deformations of a sample (e.g. darker or brighter images) will allow the network to be able to recognize similarly brighter or darker images (relative to your original training images) when deployed at testing time.

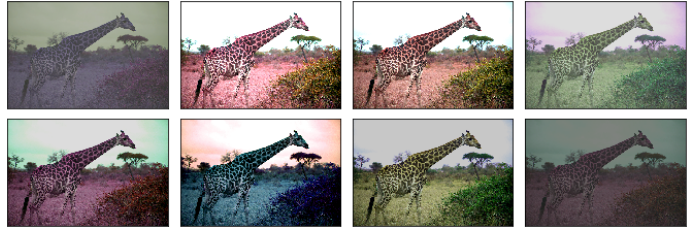


Image from MxNet

This is very important, in particular for finetuning over small datasets.
There are two big important things which you are not going to learn when trying to get experience on MNIST and CIFAR, but which matter a lot in practice on real datasets: (1) data augmentation and (2) finetuning.

- **Problem 6: Getting the scale of relevant objects right:** another example of a fixed, non-randomized augmentation for images (will see that in a later lecture):
  make the image size such that relevant structures (e.g. cats, cars) can be identified well.

  A neural network for images has convolutional kernels with a fixed size.

  The sizes of input images at test time must be such that ... the structures to be identified at test time (e.g. cats, cars) – have similar size as – structures used at training time.

If your training set contains only screen filling cat faces,

then do not complain that you will not be able to predict cat correctly on this:

Your localized convolutional kernels were simply not trained to find structures on such small scales!
but data augmentation (or training a bounding box detector) can help with that – you can classify over many small windows.

- **Above problems are not image-specific.** They occur also in NLP and other data types.
- The same problems (normalization, input dimensionality, multiple views, slightly augmented input data) arise when using a neural network e.g. for text processing, as the number of words is varying. Later we will see recurrent neural networks which can deal with inputs being sequences of arbitrary lengths (however even then it can make a big difference to deal with dimensionality properly).
- **Coding/homework:** You will use a pretrained deep neural net to compute predictions for images.

What inputs does a neural network expect ?

- As for images, common image readers produce subpixels in $[0, 255]$. The output of `PIL.Image` converted to a numpy array is usually of shape $(\mathit{height}, \mathit{width}, \underbrace{\mathit{channels}}_{\text{index: 2}})$.
- A neural network expects inputs often in $[0, 1]$ with shape $(\mathit{batchsize}, \underbrace{\mathit{channels}}_{\text{index: 1}}, \mathit{height}, \mathit{width})$ with afterwards subtracted the mean of the training dataset.
- For the conversion from $[0, 255]$ to $[0, 1]$, and switching the channel axis – in pytorch and mxnet `transforms.ToTensor()` takes care of that.

Often one has to do further preprocessing (than only conversion of scales to $[0, 1]$). The reason is:

> You have to do the same preprocessing steps at test time, as what was done at training time, otherwise train and test data have different distributions in input space. This usually results in heavily reduced performance.

- Neural nets are usually trained with images, from which the training dataset mean is subtracted (that is: for every pixel). It was observed that training with samples which have on average over the dataset a zero mean results in faster convergence. See http://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf for an explanation.
- Furthermore, often input dimensions are also normalized by dividing them over a standard deviation (either channel-wise or pixel-wise) estimated over the training set.

$$subpixel[channel] = \frac{subpixel[channel] - mean_{train}[channel]}{std_{train}[channel]}$$

$$subpixel[channel] = subpixel[channel] - mean_{train}[channel]$$

in code:

```
data_transforms['val']=transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
```

In keras, that hosts many different modes, one can see at least three different modes:

https://github.com/keras-team/keras-applications/blob/master/keras_applications/imagenet_utils.py

**The rule:**

Check what preprocessing was used at training time for the network you are going to use. Check what of that makes sense to be used at test time. Fail here $\Rightarrow$ even prediction at test time with a pretrained net will fail to reproduce test data performance .

# Problem 2: data input size vs nn input size without changing the network input size



- neural networks are trained with a certain image size
- requirement: images within one minibatch must have the same size

Most pretrained ResNets have an input size of $224 \times 224$ for images. Your images usually are not of that size.

What can one do? One can resize the images for every image and every minibatch to the same size.

# Problem 2: data input size vs nn input size without changing the network input size



What can one do? One can resize the images for every image and every minibatch to the same size.

manually composed, Centercrop:

```
data_transforms['val']=transforms.Compose([
        transforms.Resize(256), # resize smaller dim of (h,w) to 256,
        transforms.CenterCrop(224), # (224,224) crop
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
```

- can get the used transforms for torchvision model zoo models:

```
# for torchvision model zoo models: grabs the default transforms of a pretrained network
transforms = torchvision.models.EfficientNet_B0_Weights.DEFAULT.transforms()
```

# Problem 2: data input size vs nn input size without changing the network input size



What can one do? One can resize the images for every image and every minibatch to the same size.

manually composed, randomly positioned crop:

```
data_transforms['train']=transforms.Compose([
        transforms.Resize(256), # resize smaller dim of (h,w) to 256,
        transforms.RandomCrop(224), # (224,224) crop
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
```

- can get the used transforms for torchvision model zoo models:

```
# for torchvision model zoo models: grabs the default transforms of a pretrained network
transforms = torchvision.models.EfficientNet_B0_Weights.DEFAULT.transforms()
```

Most of these transforms can use pytorch tensors or PIL.Image objects.

Alternative is to resize the image to $224 \times 224$ while giving up the aspect ratio (height to width)

# Problem 2: data input size vs nn input size without changing the network input size



**Common resizing procedure**

- resize the image while preserving the aspect ratio such that the smaller size $s$ is $s \geq S + K$, (example $S = 224$, $K = 32$) pixels, and then take a crop of size $S \times S$
- the crop can have multiple approaches: a **center crop** or a **random crop** of $S \times S$ with random position.

The same problem occurs in sequence analysis with 1d-cnns - what sequence length to input ?

Recurrent neural networks can deal with sequences of arbitrary lengths.

Can you change the input size for a pretrained network composed only of `nn.Linear`, activation and pooling layers?

# Problem 3: changing the network input size



You want to classify over a larger input than $224 \times 224$, say $330 \times 360$ ?

Can you change the input size for a pretrained network composed only of `nn.Conv2d`, activation and pooling layers?

# Problem 3: changing the network input size

You want to classify over a larger input than $224 \times 224$, say $330 \times 360$ ?

Can you change the input size for a pretrained network composed only of `nn.Conv2d`, activation and pooling layers?

- The important thing to understand is: if you change the input size, then this does **NOT** affect the number of parameters in any convolutional layer, only its output sizes. Using a pretrained network is fine wrt the `nn.Conv2d` layers.
- Isn't there an issue with the last linear layer ?

You want to classify over a larger input than $224 \times 224$, say $330 \times 360$ ?

Can you change the input size for a pretrained network composed only of `nn.Conv2d`, activation and pooling layers?

- The important thing to understand is: if you change the input size, then this does **NOT** affect the number of parameters in any convolutional layer, only its output sizes. Using a pretrained network is fine wrt the `nn.Conv2d` layers.
- Isn't there an issue with the last linear layer ?

Yes you can usually do that but ...

- The idea can break at subsequent fully connected layers - but often it can be fixed. So if there is a fully connected layer, with a pooling or flatten layer before it, then one must make sure that it is a global pooling with a fixed output size.

Lets look at a resnet-18 to understand this:
https://github.com/pytorch/vision/blob/master/torchvision/models/resnet.py in `class ResNet(nn.Module):` and `def resnet18(pretrained=False, **kwargs):`
Consider how the forward pass works `def forward(self,x):`

$$(224, 224) \rightarrow \text{many conv layers} \rightarrow (512, 7, 7) \rightarrow \textbf{Global Pooling by } \texttt{self.avgpool}$$
$$\rightarrow (512, 1, 1) \rightarrow x = x.view(x.size(0), -1) \rightarrow (512) \rightarrow self.fc- > (1000)$$

```
self.avgpool = nn.AdaptiveAvgPool2d(...)
```

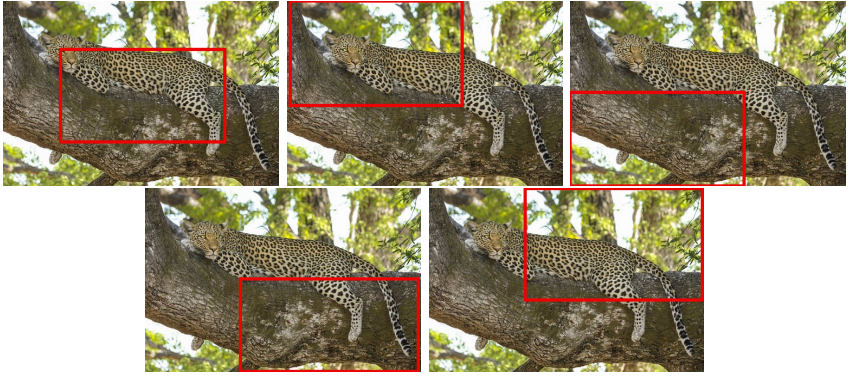
**when resizing input to a network**

- Convolution layers: Resizing the input of a neural network changes the sizes of resulting feature maps, but for convolution layers it does **not change their number of parameters** – therefore loading pretrained convolution layers will still work.
- Dense Layers: Resizing the input of a neural network **changes the number of parameters** in fully connected/dense layers. In order to ensure the same number of parameters in fully connected layers with changed size as with original input size, one needs to ensure that the number of inputs to fully connected layers stays the same. If the fully connected layer is preceded by a pooling layer, then the pooling layer can be modified into a global pooling with fixed output size to solve this problem.
- Useful: Global pooling/adaptive pooling are pooling methods where the output size is fixed and instead the pooling kernel size is changing dependent on the input to match the desired output size.

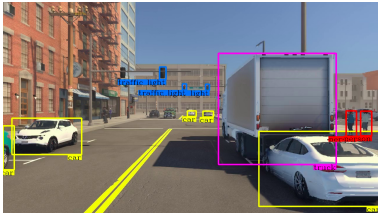
https://pytorch.org/vision/stable/transforms.html

Examples:

- Center Crop, Corner Crop, RandomCrop
  https://pytorch.org/vision/main/generated/torchvision.transforms.RandomCrop.html



- in some problems the data $(x, y)$ has spatial information in its ground truth labels, for example in object detection



credit: https://developer.nvidia.com/blog/deploying-a-scalable-object-detection-pipeline-the-inferencin

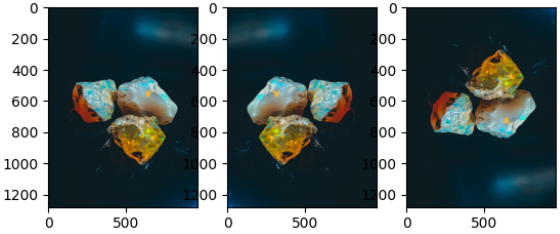
**BUG/MISTAKE Risk**

- if your data pair $(x, y)$ has ground truth labels $y$ which contain spatial information in them, then you must transform $y$ in the same way as $x$ for any geometric transform, e.g. for cropping, rotation, mirroring

https://pytorch.org/blog/extending-torchvisions-transforms-to-object-detection-segmentation-and-video-tasks/

Examples:

- Random Horizontal and vertical flip

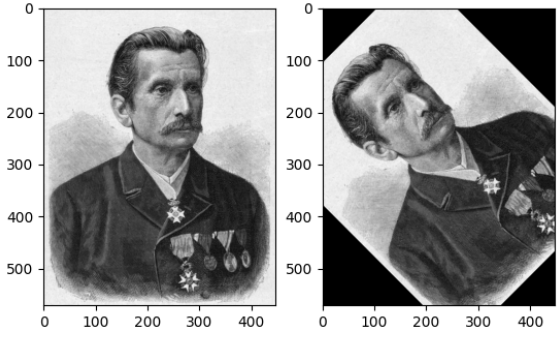



...

```
img_t2 = torchvision.transforms.functional.hflip(img_t)
img_t3 = torchvision.transforms.functional.vflip(img_t)
```

Examples:

- Rotations



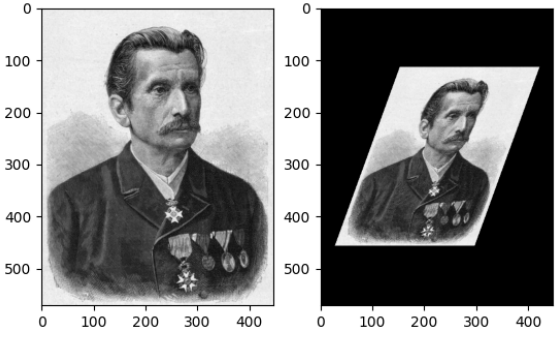

...

```
img_t2 =torchvision.transforms.functional.rotate( img_t,angle=45 )
```

Examples:

- Affine transforms (can be rotation, scaling , shearing, shifting)

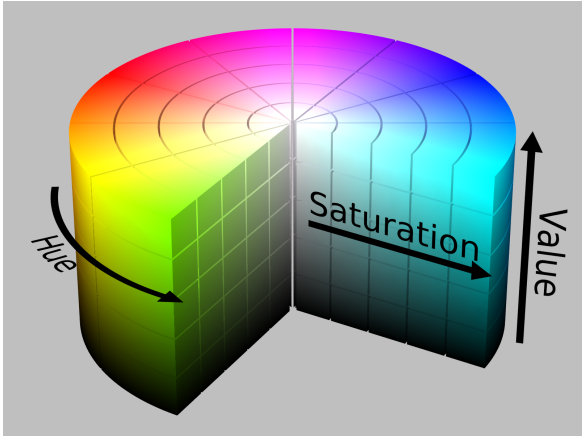
...




```
img_t2 =torchvision.transforms.functional.affine( img_t,angle=0 , translate = (0,0), scale = 0.6, shear = (20,0) )
```

Examples:
Color Jitter in HSV/HSL Color space

- HSV:
  - Hue (color type) - is cyclic! https://en.wikipedia.org/wiki/HSL_and_HSV
  - saturation: intensity of color
  - value: brightness
- color jitter applies an additive changes among each of these axes.




SharkD@wikipedia https://commons.wikimedia.org/w/index.php?curid=9801673

Examples:
Color Jitter in HSV/HSL Color space

- HSV:
  - Hue (color type) - is cyclic! https://en.wikipedia.org/wiki/HSL_and_HSV
  - saturation: intensity of color
  - value: brightness
- Randomized color jitter applies an additive changes among each of these axes.

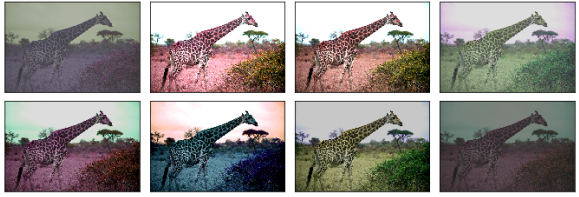
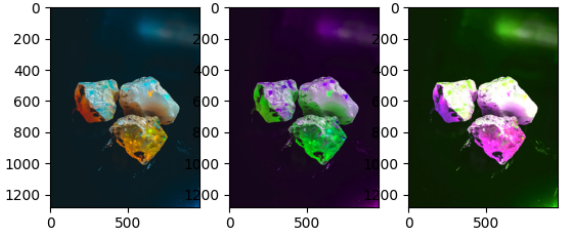


Image from MxNet

Examples:

- Random Colorjitter in Pytorch:

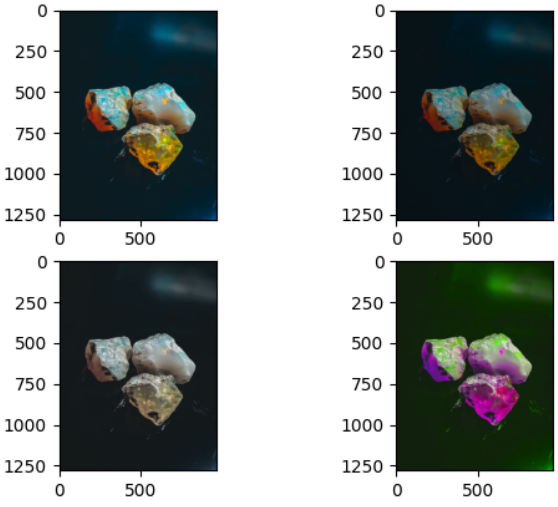



...

```
trf = torchvision.transforms.ColorJitter(brightness=0.5,contrast=0.5,saturation=0.5,hue=0.3)
img_t2 = trf(img_t)
```

Examples:

- fixed hue/saturation/brightness adjustments in Pytorch:




```
img_t4 = torchvision.transforms.functional.adjust_brightness(img_t, 0.7) #[0, \infty]
img_t3 = torchvision.transforms.functional.adjust_saturation(img_t, 0.7) #[0, \infty]
img_t2 = torchvision.transforms.functional.adjust_hue(img_t,-0.25) #[-0.5,0.5]
```

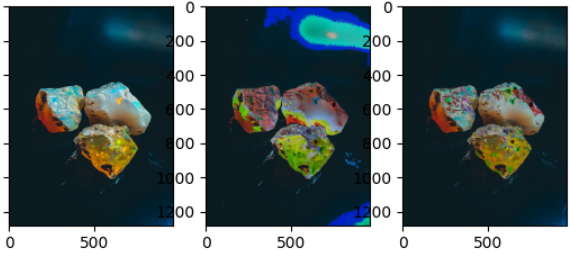
Examples:

Brightness , Contrast - for random augmentations again `ColorJitter`:

```
trf = torchvision.transforms.ColorJitter(brightness=0.5,contrast=0.5,saturation=0.0,hue=0.0)
img_t2 = trf(img_t)
```

Examples:

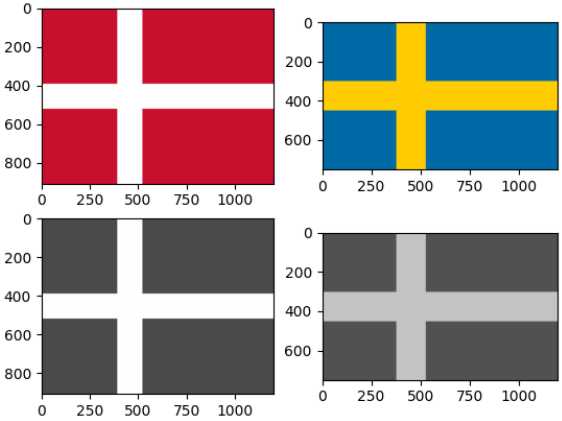
- Solarization: invert colors above a certain brightness threshold




```
trf1 = torchvision.transforms.RandomSolarize(threshold=0.2,p=1.0)
trf2 = torchvision.transforms.RandomSolarize(threshold=0.75,p=1.0)
img_t2 = trf1(img_t)
img_t3 = trf2(img_t)
```

Examples:

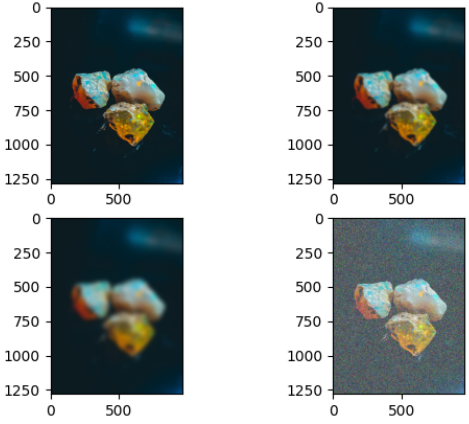
- Grayscaling




```
trf1 = torchvision.transforms.Grayscale(num_output_channels=3)
img_t3 = trf1(img_t)
```

Examples:

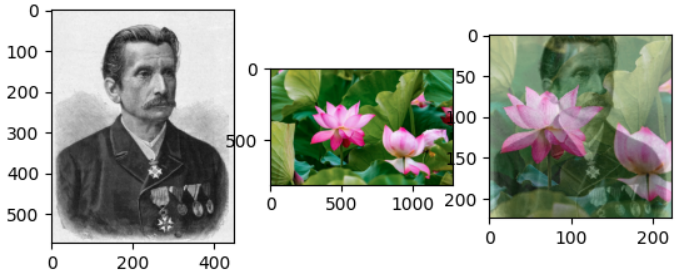
- Blur
- Adding noise (many types of noise exist!)




```
img_t2 = torchvision.transforms.functional.gaussian_blur(img_t, [31,31]) # kernel size
```

Mixup https://arxiv.org/pdf/1710.09412.pdf

- draw randomly a mixing parameter $\lambda$ (from a Beta distribution)
- blend 2 images together into $x$ with mixing parameter $\lambda$
- must have same shape (Crop!)
- blend label $y$ with the same factor $\lambda$. Not 0-1-labels anymore




$$\lambda \sim Beta(a, a) \in (0, 1)$$
$$x = \lambda x_0 + (1 - \lambda) x_1$$
$$y = \lambda y_0 + (1 - \lambda) y_1$$

Cutmix https://arxiv.org/pdf/1905.04899.pdf

- big idea: cutout a random section from one image and transplant it into another
- draw randomly a mixing parameter $\lambda$
- draw randomly bounding box coordinates $(r_x, r_y, r_w, r_h)$ so that cutout area ratio is $\frac{r_w r_h}{HW} = 1 - \lambda$
- blend label $y$ with the same factor $\lambda$. Not 0-1-labels anymore.

$$\lambda \sim Beta(a, a) \in (0, 1)$$
$$r_w = W\sqrt{1-\lambda},\ r_h = H\sqrt{1-\lambda},\ \text{... so that } \frac{r_w r_h}{HW} = 1 - \lambda$$
$$r_x \sim \text{Unif}(0, W - W\sqrt{1-\lambda}),\ r_y \sim \text{Unif}(0, H - H\sqrt{1-\lambda})$$
$$(r_x, r_y, r_w, r_h) \mapsto \text{binary mask } M \text{ (0 where the left } x_0 \text{ will receive the transplant)}$$
$$x = M \odot x_0 + (1 - M) \odot x_1$$
$$y = \lambda y_0 + (1 - \lambda) y_1$$

More transformations exist.

You should be able to categorize:

- Geometric transformations (change in spatial relationships or size)
- vs Photometric transformations (global change of photoreceptive properties)
- vs "both categories" (Blur)
- vs other categories

Some transformations are unsuitable for some problems!

- The idea of data augmentation at test time is to predict on multiple views of the same input. This may help in cases with high uncertainty or with missing views (see homework!).

Examples of data augmentation at test time for images:

- random crops of a fixed size of the image
- corner and center crops
- mirroring along the vertical axis

**in pytorch?**

https://pytorch.org/vision/stable/transforms.html

- The idea of data augmentation at test time is to predict on multiple views of the same input.

**Test time data augmentation**

Let $x$ be an input sample, $f(\cdot)$ a predictor, and $A(a_i, x)$ the i-th augmentation with parameter $a_i$ (e.g. a crop of an image, or a change in brightness), then the prediction is computed as an average over a set of chosen augmentations:

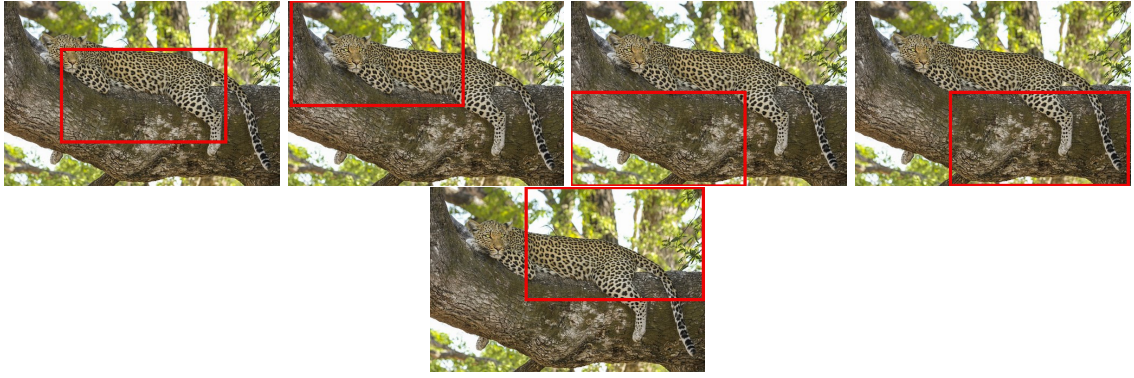
$$f_{avg}(x) = \frac{1}{n_A} \sum_{i=1}^{n_A} f(A(a_i, x)), \text{ param } a_i \text{ deterministic or } a_i \sim P$$

The average is computed over augmentations of the same input sample $x$.

Think: what can the parameters $a_i$ be ?

- The idea of data augmentation at test time is to predict on multiple views of the same input.

prediction $= \frac{1}{5}$( f(Center crop)+ f(upper left crop)+ f(lower left )+ f(lower right)+ f(lower right crop) )



## Training time data augmentation

The idea of data augmentation at training time is to extend the training data available by creating instances

- which can be **plausibly** derived from the training data set
- which are plausible to have the same ground truth label as their origin, or where the label can be derived meaningfully
- which are plausible to be seen in the test set.
- which sometimes are used to encode invariances that one wants to have in the learning process (e.g. recognition independent of whether object is rotated ... (!))

Thus to increase the training dataset.

$$f(x) = f(A_a(x)), \ a \sim P_\theta(a)$$

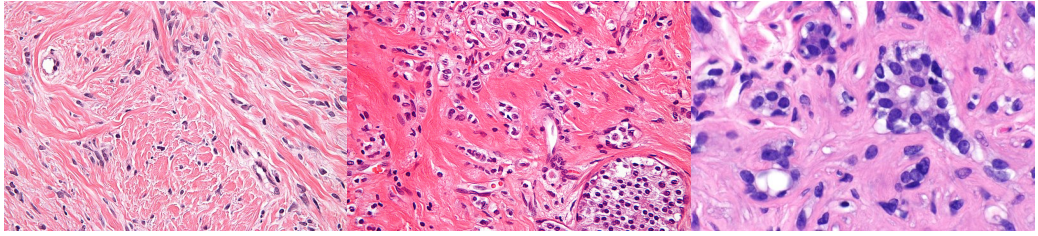
**Learning theory/Generalization perspective: Data augmentation vs the iid assumption???**

Which of the above augmentations are suitable or less suitable for Training ?

Which of the above augmentations are suitable or less suitable for Training ?

There might be some problem specific limitations:

An example where playing with hues a little bit may make sense: Below is the same stain Haematoxylin and Eosin, of the same tissue. Different lab people create different color intensities!



## General data augmentation

is a transformation of an input $x$

$$A(x)$$

Often it has a parameter $a$, e.g. value of brightness to add

$$A_a(x)$$

Such parameters, are at training time often drawn from a random distribution

$$a \sim P_\theta(a)$$

The parameters $\theta$ of the random distribution are hyperparameters, and need to be validated on a validation set.

**Augment to what degree?**

Data augmentation methods depends typically on some parameter. One needs to

- train with multiple choices of the parameter,
- find the best parameter by looking at validation dataset errors.
- Once all parameters have been selected, then one does one final evaluation on the test data set – in order to check that one did not overfit by optimizing parameters by looking at the validation error.

particularly important if

- training from scratch
- model destillation / teacher-student training

Next Question: How to optimize all those hyperparameters ??

Cubuk et al. 2019 https://arxiv.org/abs/1909.13719

Given 14 transforms:

```
transforms = ['Identity', 'AutoContrast', 'Equalize','Rotate', 'Solarize',
 'Color', 'Posterize','Contrast', 'Brightness', 'Sharpness',
 'ShearX', 'ShearY', 'TranslateX', 'TranslateY']
```

- $+$ given hyperparameters $N, M$:
  - $N$ number of transformations applied to one sample sequentially
  - $M$ intensity of distortion
- do not care about order of them or which ones
- randomly draw any $N$ with uniform probability*!
- define strength of each augmentation by an integer $0, \ldots, 10$ by linearly partitioning a maximal range (based on Table 6 in Cubuk et al.https://arxiv.org/abs/1805.09501)
  - apply each transformation with the same strength $M \in 0, \ldots, 10$
- optimize only over hyperparameters $M, N$

Cubuk et al. 2019 https://arxiv.org/abs/1909.13719

Given 14 transforms, given hyperparameters $N, M$:

```
transforms = ['Identity', 'AutoContrast', 'Equalize','Rotate', 'Solarize',
 'Color', 'Posterize','Contrast', 'Brightness', 'Sharpness',
 'ShearX', 'ShearY', 'TranslateX', 'TranslateY']
```

- randomly draw any $N$ with uniform probability*!
- define strength of each augmentation by an integer $0, \ldots, 10$ by linearly partitioning a maximal range (based on Table 6 in Cubuk et al.https://arxiv.org/abs/1805.09501)
  - apply each transformation with the same strength $M \in 0, \ldots, 10$
- optimize only over hyperparameters $M, N$

In result almost as good as searching for specific policies (e.g Table 4 in the paper, factor $10^{32}$ faster).

(* see section 4.7 in the paper – tried optimization with $\alpha_{ij}$ - the probability to use transformation $i$ in step $j$ for $N = 2$ – expensive, needs $14N$ transformations to be tried out on every image.)

**A word of caution**

These 14 transformations were found to be useful on general ImageNet images.
This may not hold for other domains, eg. medical or satellite images. It may not hold when your domain has much fewer images than imagenet.

This is a general topic in machine learning, not limited to computer vision!

- generally: add noise to features, labels
- use generative methods to create similar samples (risk (!!): undersampling of rare regions, or that generative methods lie concentrated in some regions of target network feature spaces, mode collapse in feature space), hallucinatory samples
- nlp: replace words by synonyms/similar words. translate to another language and back.
- audio: modify speed (locally, globally)